

A photograph of a nuclear power plant with two large cooling towers emitting thick white steam into a clear blue sky. The plant is situated near a body of water, which reflects the towers and the sky. In the foreground, there are some green trees and a grassy bank.

Delta-debug : opportunité pour des supports d'exécution parallèle.

Seminaire Causerie Scientifique
30/01/2025

Bruno Lathuilière

EDF R&D



Muller example

```
float muller(int nt, bool verbose=false){
    float x0 = 11./2.;
    float x1 = 61./11.;
    for(size_t it=0; it < nt ; it++){
        float tmp0 = 3000./x0;
        float tmp1 = 1130. - tmp0;
        float tmp2 = tmp1 /x1 ;
        float x2 = 111. - tmp2;
        if( verbose ){
            cout <<"it: "<< it << " x2: "<<x2 << " tmp0: "<<tmp0 <<" tmp1: "<<tmp1<<" tmp2: "<<tmp2 << endl;
        }
        x0 = x1;
        x1 = x2;
    }
    std::cout <<"x["<<nt<<"]="<<x1<<std::endl;
    return x1;
}
```

./muller -v > stdout_nearest

stdout_nearest

```
it: 0 x2: 5.59016 tmp0: 545.455 tmp1: 584.545 tmp2: 105.410
it: 1 x2: 5.63343 tmp0: 540.984 tmp1: 589.016 tmp2: 105.367
it: 2 x2: 5.67465 tmp0: 536.657 tmp1: 593.343 tmp2: 105.325
it: 3 x2: 5.71333 tmp0: 532.535 tmp1: 597.465 tmp2: 105.287
it: 4 x2: 5.74912 tmp0: 528.667 tmp1: 601.333 tmp2: 105.251
it: 5 x2: 5.78181 tmp0: 525.088 tmp1: 604.912 tmp2: 105.218
it: 6 x2: 5.81131 tmp0: 521.819 tmp1: 608.181 tmp2: 105.189
it: 7 x2: 5.83766 tmp0: 518.869 tmp1: 611.131 tmp2: 105.162
it: 8 x2: 5.86108 tmp0: 516.234 tmp1: 613.766 tmp2: 105.139
it: 9 x2: 5.88354 tmp0: 513.904 tmp1: 616.096 tmp2: 105.116
it: 10 x2: 5.93596 tmp0: 511.851 tmp1: 618.149 tmp2: 105.064
it: 11 x2: 6.53442 tmp0: 509.897 tmp1: 620.103 tmp2: 104.466
x[12]=6.53442
```

Muller example

```
float muller(int nt, bool verbose=false){
    float x0 = 11./2.;
    float x1 = 61./11.;
    for(size_t it=0; it < nt ; it++){
        float tmp0 = 3000./x0;
        float tmp1 = 1130. - tmp0;
        float tmp2 = tmp1 /x1 ;
        float x2 = 111. - tmp2;
        if( verbose ){
            cout <<"it: "<< it << " x2: "<<x2 << " tmp0: "<<tmp0 <<" tmp1: "<<tmp1<<" tmp2: "<<tmp2 << endl;
        }
        x0 = x1;
        x1 = x2;
    }
    std::cout <<"x["<<nt<<"]="<<x1<<std::endl;
    return x1;
}
```

./muller -v > stdout_nearest

valgrind --tool=verrou --rounding-mode=random ./muller -v > stdout_random_1

stdout_nearest	stdout_random_1
it: 0 x2: 5.59016 tmp0: 545.455 tmp1: 584.545 tmp2: 105.410	it: 0 x2: 5.59016 tmp0: 545.455 tmp1: 584.545 tmp2: 105.410
it: 1 x2: 5.63343 tmp0: 540.984 tmp1: 589.016 tmp2: 105.367	it: 1 x2: 5.63343 tmp0: 540.984 tmp1: 589.016 tmp2: 105.367
it: 2 x2: 5.67465 tmp0: 536.657 tmp1: 593.343 tmp2: 105.325	it: 2 x2: 5.67465 tmp0: 536.657 tmp1: 593.343 tmp2: 105.325
it: 3 x2: 5.71333 tmp0: 532.535 tmp1: 597.465 tmp2: 105.287	it: 3 x2: 5.71333 tmp0: 532.535 tmp1: 597.465 tmp2: 105.287
it: 4 x2: 5.74912 tmp0: 528.667 tmp1: 601.333 tmp2: 105.251	it: 4 x2: 5.74912 tmp0: 528.667 tmp1: 601.333 tmp2: 105.251
it: 5 x2: 5.78181 tmp0: 525.088 tmp1: 604.912 tmp2: 105.218	it: 5 x2: 5.78181 tmp0: 525.088 tmp1: 604.912 tmp2: 105.218
it: 6 x2: 5.81131 tmp0: 521.819 tmp1: 608.181 tmp2: 105.189	it: 6 x2: 5.81132 tmp0: 521.819 tmp1: 608.181 tmp2: 105.189
it: 7 x2: 5.83766 tmp0: 518.869 tmp1: 611.131 tmp2: 105.162	it: 7 x2: 5.83767 tmp0: 518.869 tmp1: 611.131 tmp2: 105.162
it: 8 x2: 5.86108 tmp0: 516.234 tmp1: 613.766 tmp2: 105.139	it: 8 x2: 5.86120 tmp0: 516.234 tmp1: 613.766 tmp2: 105.139
it: 9 x2: 5.88354 tmp0: 513.904 tmp1: 616.096 tmp2: 105.116	it: 9 x2: 5.88567 tmp0: 513.904 tmp1: 616.096 tmp2: 105.114
it: 10 x2: 5.93596 tmp0: 511.851 tmp1: 618.149 tmp2: 105.064	it: 10 x2: 5.97214 tmp0: 511.840 tmp1: 618.160 tmp2: 105.028
it: 11 x2: 6.53442 tmp0: 509.897 tmp1: 620.103 tmp2: 104.466	it: 11 x2: 7.13639 tmp0: 509.712 tmp1: 620.288 tmp2: 103.864
x[12]=6.53442	x[12]=7.13639

Muller example

```
float muller(int nt, bool verbose=false){
    float x0 = 11./2.;
    float x1 = 61./11.;
    for(size_t it=0; it < nt ; it++){
        float tmp0 = 3000./x0;
        float tmp1 = 1130. - tmp0;
        float tmp2 = tmp1 /x1 ;
        float x2 = 111. - tmp2;
        if( verbose ){
            cout <<"it: "<< it << " x2: "<<x2 << " tmp0: "<<tmp0 <<" tmp1: "<<tmp1<<" tmp2: "<<tmp2 << endl;
        }
        x0 = x1;
        x1 = x2;
    }
    std::cout <<"x["<<nt<<"]="<<x1<<std::endl;
    return x1;
}
```

```
./muller -v > stdout_nearest
valgrind -tool=verrou -rounding-mode=random ./muller -v > stdout_random_1
valgrind -tool=verrou -rounding-mode=random ./muller -v > stdout_random_2
```

stdout_nearest	stdout_random_1	stdout_random_2
it: 0 x2: 5.59016 tmp0: 545.455 tmp1: 584.545 tmp2: 105.410	it: 0 x2: 5.59016 tmp0: 545.455 tmp1: 584.545 tmp2: 105.410	it: 0 x2: 5.59016 tmp0: 545.455 tmp1: 584.545 tmp2: 105.410
it: 1 x2: 5.63343 tmp0: 540.984 tmp1: 589.016 tmp2: 105.367	it: 1 x2: 5.63343 tmp0: 540.984 tmp1: 589.016 tmp2: 105.367	it: 1 x2: 5.63343 tmp0: 540.984 tmp1: 589.016 tmp2: 105.367
it: 2 x2: 5.67465 tmp0: 536.657 tmp1: 593.343 tmp2: 105.325	it: 2 x2: 5.67465 tmp0: 536.657 tmp1: 593.343 tmp2: 105.325	it: 2 x2: 5.67465 tmp0: 536.657 tmp1: 593.343 tmp2: 105.325
it: 3 x2: 5.71333 tmp0: 532.535 tmp1: 597.465 tmp2: 105.287	it: 3 x2: 5.71333 tmp0: 532.535 tmp1: 597.465 tmp2: 105.287	it: 3 x2: 5.71333 tmp0: 532.535 tmp1: 597.465 tmp2: 105.287
it: 4 x2: 5.74912 tmp0: 528.667 tmp1: 601.333 tmp2: 105.251	it: 4 x2: 5.74912 tmp0: 528.667 tmp1: 601.333 tmp2: 105.251	it: 4 x2: 5.74912 tmp0: 528.667 tmp1: 601.333 tmp2: 105.251
it: 5 x2: 5.78181 tmp0: 525.088 tmp1: 604.912 tmp2: 105.218	it: 5 x2: 5.78181 tmp0: 525.088 tmp1: 604.912 tmp2: 105.218	it: 5 x2: 5.78181 tmp0: 525.088 tmp1: 604.912 tmp2: 105.218
it: 6 x2: 5.81131 tmp0: 521.819 tmp1: 608.181 tmp2: 105.189	it: 6 x2: 5.81131 tmp0: 521.819 tmp1: 608.181 tmp2: 105.189	it: 6 x2: 5.81131 tmp0: 521.819 tmp1: 608.181 tmp2: 105.189
it: 7 x2: 5.83766 tmp0: 518.869 tmp1: 611.131 tmp2: 105.162	it: 7 x2: 5.83765 tmp0: 518.869 tmp1: 611.131 tmp2: 105.162	it: 7 x2: 5.83765 tmp0: 518.869 tmp1: 611.131 tmp2: 105.162
it: 8 x2: 5.86108 tmp0: 516.234 tmp1: 613.766 tmp2: 105.139	it: 8 x2: 5.86080 tmp0: 516.234 tmp1: 613.766 tmp2: 105.139	it: 8 x2: 5.86080 tmp0: 516.234 tmp1: 613.766 tmp2: 105.139
it: 9 x2: 5.88354 tmp0: 513.904 tmp1: 616.096 tmp2: 105.116	it: 9 x2: 5.87884 tmp0: 513.906 tmp1: 616.094 tmp2: 105.121	it: 9 x2: 5.87884 tmp0: 513.906 tmp1: 616.094 tmp2: 105.121
it: 10 x2: 5.93596 tmp0: 511.851 tmp1: 618.149 tmp2: 105.064	it: 10 x2: 5.85595 tmp0: 511.875 tmp1: 618.125 tmp2: 105.144	it: 10 x2: 5.85595 tmp0: 511.875 tmp1: 618.125 tmp2: 105.144
it: 11 x2: 6.53442 tmp0: 509.897 tmp1: 620.103 tmp2: 104.466	it: 11 x2: 5.17686 tmp0: 510.305 tmp1: 619.695 tmp2: 105.823	it: 11 x2: 5.17686 tmp0: 510.305 tmp1: 619.695 tmp2: 105.823
x[12]=6.53442	x[12]=5.17686	x[12]=5.17686

Muller example

```
float muller(int nt, bool verbose=false){
    float x0 = 11./2.;
    float x1 = 61./11.;
    for(size_t it=0; it < nt ; it++){
        float tmp0 = 3000./x0;
        float tmp1 = 1130. - tmp0;
        float tmp2 = tmp1 /x1 ;
        float x2 = 111. - tmp2;
        if( verbose ){
            cout <<"it: "<< it << " x2: "<<x2 << " tmp0: "<<tmp0 <<" tmp1: "<<tmp1<<" tmp2: "<<tmp2 << endl;
        }
        x0 = x1;
        x1 = x2;
    }
    std::cout <<"x["<<nt<<"]="<<x1<<std::endl;
    return x1;
}
```

```
valgrind -tool=verrou -rounding-mode=random ./muller -v > stdout_nearest
valgrind -tool=verrou -rounding-mode=random ./muller -v > stdout_random_1
valgrind -tool=verrou -rounding-mode=random ./muller -v > stdout_random_2
valgrind -tool=verrou -rounding-mode=random ./muller -v > stdout_random_3
```

stdout_nearest	stdout_random_1	stdout_random_2	stdout_random_3
it: 0 x2: 5.59016 tmp0: 545.455 tmp1: 584.545 tmp2: 105.410	it: 0 x2: 5.59016 tmp0: 545.455 tmp1: 584.545 tmp2: 105.410	it: 0 x2: 5.59016 tmp0: 545.455 tmp1: 584.545 tmp2: 105.410	it: 0 x2: 5.59016 tmp0: 545.455 tmp1: 584.545 tmp2: 105.410
it: 1 x2: 5.63343 tmp0: 540.984 tmp1: 589.016 tmp2: 105.367	it: 1 x2: 5.63343 tmp0: 540.984 tmp1: 589.016 tmp2: 105.367	it: 1 x2: 5.63343 tmp0: 540.984 tmp1: 589.016 tmp2: 105.367	it: 1 x2: 5.63343 tmp0: 540.984 tmp1: 589.016 tmp2: 105.367
it: 2 x2: 5.67465 tmp0: 536.657 tmp1: 593.343 tmp2: 105.325	it: 2 x2: 5.67465 tmp0: 536.657 tmp1: 593.343 tmp2: 105.325	it: 2 x2: 5.67465 tmp0: 536.657 tmp1: 593.343 tmp2: 105.325	it: 2 x2: 5.67465 tmp0: 536.657 tmp1: 593.343 tmp2: 105.325
it: 3 x2: 5.71333 tmp0: 532.535 tmp1: 597.465 tmp2: 105.287	it: 3 x2: 5.71333 tmp0: 532.535 tmp1: 597.465 tmp2: 105.287	it: 3 x2: 5.71333 tmp0: 532.535 tmp1: 597.465 tmp2: 105.287	it: 3 x2: 5.71333 tmp0: 532.535 tmp1: 597.465 tmp2: 105.287
it: 4 x2: 5.74912 tmp0: 528.667 tmp1: 601.333 tmp2: 105.251	it: 4 x2: 5.74912 tmp0: 528.667 tmp1: 601.333 tmp2: 105.251	it: 4 x2: 5.74912 tmp0: 528.667 tmp1: 601.333 tmp2: 105.251	it: 4 x2: 5.74912 tmp0: 528.667 tmp1: 601.333 tmp2: 105.251
it: 5 x2: 5.78181 tmp0: 525.088 tmp1: 604.912 tmp2: 105.218	it: 5 x2: 5.78181 tmp0: 525.088 tmp1: 604.912 tmp2: 105.218	it: 5 x2: 5.78181 tmp0: 525.088 tmp1: 604.912 tmp2: 105.218	it: 5 x2: 5.78181 tmp0: 525.088 tmp1: 604.912 tmp2: 105.218
it: 6 x2: 5.81131 tmp0: 521.819 tmp1: 608.181 tmp2: 105.189	it: 6 x2: 5.81131 tmp0: 521.819 tmp1: 608.181 tmp2: 105.189	it: 6 x2: 5.81131 tmp0: 521.819 tmp1: 608.181 tmp2: 105.189	it: 6 x2: 5.81131 tmp0: 521.819 tmp1: 608.181 tmp2: 105.189
it: 7 x2: 5.83766 tmp0: 518.869 tmp1: 611.131 tmp2: 105.162	it: 7 x2: 5.83766 tmp0: 518.869 tmp1: 611.131 tmp2: 105.162	it: 7 x2: 5.83766 tmp0: 518.869 tmp1: 611.131 tmp2: 105.162	it: 7 x2: 5.83766 tmp0: 518.869 tmp1: 611.131 tmp2: 105.162
it: 8 x2: 5.86108 tmp0: 516.234 tmp1: 613.766 tmp2: 105.139	it: 8 x2: 5.86108 tmp0: 516.234 tmp1: 613.766 tmp2: 105.139	it: 8 x2: 5.86108 tmp0: 516.234 tmp1: 613.766 tmp2: 105.139	it: 8 x2: 5.86108 tmp0: 516.234 tmp1: 613.766 tmp2: 105.139
it: 9 x2: 5.88354 tmp0: 513.904 tmp1: 616.096 tmp2: 105.116	it: 9 x2: 5.88355 tmp0: 513.904 tmp1: 616.096 tmp2: 105.116	it: 9 x2: 5.88355 tmp0: 513.904 tmp1: 616.096 tmp2: 105.116	it: 9 x2: 5.88355 tmp0: 513.904 tmp1: 616.096 tmp2: 105.116
it: 10 x2: 5.93596 tmp0: 511.851 tmp1: 618.149 tmp2: 105.064	it: 10 x2: 5.93607 tmp0: 511.851 tmp1: 618.149 tmp2: 105.064	it: 10 x2: 5.93607 tmp0: 511.851 tmp1: 618.149 tmp2: 105.064	it: 10 x2: 5.93607 tmp0: 511.851 tmp1: 618.149 tmp2: 105.064
it: 11 x2: 6.53442 tmp0: 509.897 tmp1: 620.103 tmp2: 104.466	it: 11 x2: 6.53624 tmp0: 509.896 tmp1: 620.104 tmp2: 104.464	it: 11 x2: 6.53624 tmp0: 509.896 tmp1: 620.104 tmp2: 104.464	it: 11 x2: 6.53624 tmp0: 509.896 tmp1: 620.104 tmp2: 104.464
x[12]=6.53442	x[12]=6.53624	x[12]=6.53624	x[12]=6.53624

Delta-debug: verrou_dd_line vu par l'utilisateur

runScript: ddRun.sh



```
#!/bin/bash
PREFIX="valgrind --tool=verrou
--rounding-mode=random"
$PREFIX ./muller -v > $1/res.dat
```

cmpScript: extractOrCmp.py



```
#!/usr/bin/python3
import sys
from os.path import join
def extractValue(rep):
    for line in (open(join(rep, "res.dat")).readlines()):
        if line.startswith("x[12]="):
            return float(line.partition("=")[2])

if __name__=="__main__":
    if len(sys.argv)==2:
        print(extractValue(sys.argv[1]))
    if len(sys.argv)==3:
        valueRef=extractValue(sys.argv[1])
        value=extractValue(sys.argv[2])
        relDiff=abs((value-valueRef)/valueRef)
        if relDiff < 1.e-2: sys.exit(0) #OK
        else: sys.exit(1) #KO
```

Delta-debug search:

```
verrou_dd_line --nruns=5 ddRun.sh extractOrCmp.py
```

```
ddmin0 (
muller.cpp:14 (muller(unsigned long, bool)):
...
ddmin1 (
muller.cpp:12 (muller(unsigned long, bool)):
```

Les grandes lignes

① Génération d'un espace de recherche et d'une référence.

outil	espace de recherche
verrou_dd_sym	symboles
verrou_dd_line	lignes de code (si code compilé avec -g)
verrou_dd_task	tâches définies dans le code (manuellement ou automatiquement pour python)
verrou_dd_stdout	tâches définies via la sortie standard
vfc_ddebug	lignes de code pour code instrumenté par verifcarlo

② Vérification :

- ▶ l'ensemble de l'espace de recherche perturbé produit une erreur.
- ▶ l'ensemble de l'espace de recherche non-perturbé passe.

③ Algorithme de recherche par essai/erreur.

Définition 1-minimal / 1-maximal

On note Δ l'ensemble de recherche. On a $test(\{\}) = \checkmark$ et $test(\Delta) = \times$.
On aimerait :

$$test(\Delta_{\min}) = \times$$

$$test(\Delta_{\text{test}}) = \checkmark \quad \forall \Delta_{\text{test}} \in \text{subset}(\Delta) \quad \text{with} \quad \#\Delta_{\text{test}} < \#\Delta_{\min}$$

On va se contenter d'espaces 1-minimal.

$$test(\Delta_{1-\min}) = \times,$$

$$test(\Delta_{1-\min} - \{\delta\}) = \checkmark, \quad \forall \delta \in \Delta_{1-\min}.$$

Pour l'optimisation on définit les espaces 1-maximal.

$$test(\Delta_{1-\max}) = \checkmark,$$

$$test(\Delta_{1-\max} \cup \{\delta\}) = \times, \quad \forall \delta \in \Delta - \Delta_{1-\max}.$$

Zeller DDMIN

Data : Δ : search space

Data : g : granularity (default value 2)

$\{\Delta_1, \dots, \Delta_g\} \leftarrow \text{split}(\Delta, g)$;

for $j \in \llbracket 1, g \rrbracket$ **do**

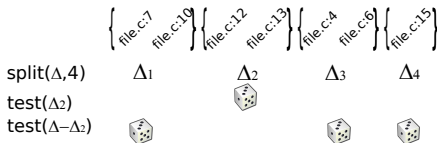
if $\text{test}(\Delta_j) = \text{X}$ **then**
 return $\text{DDmin}(\Delta_j, 2)$;

for $j \in \llbracket 1, g \rrbracket$ **do**

if $\text{test}(\Delta - \Delta_j) = \text{X}$ **then**
 return $\text{DDmin}(\Delta - \Delta_j, g - 1)$;

if $g = \#\Delta$ **then return** Δ ;

return $\text{DDmin}(\Delta, \min(2g, \#\Delta))$;



Remarques :

- ◆ $\text{assert}(\text{test}(\{\}) = \checkmark)$
- ◆ $\text{assert}(\text{test}(\Delta) = \text{X})$
- ◆ $\text{split}(\Delta - \Delta_j, g - 1) = \{\Delta_i \mid i \neq j\}$
- ◆ on peut commencer avec une granularité supérieure à 2.
- ◆ on peut ignorer la première boucle.
- ◆ l'implémentation de Zeller casse la récursion.
- ◆ on a besoin d'un cache.

Zeller DDMIN

Data : Δ : search space

Data : g : granularity (default value 2)

$\{\Delta_1, \dots, \Delta_g\} \leftarrow \text{split}(\Delta, g)$;

for $j \in \llbracket 1, g \rrbracket$ **do**

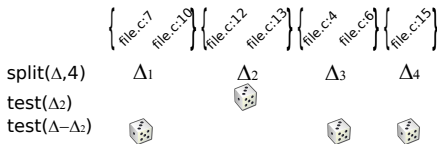
if $\text{test}(\Delta_j) = \text{X}$ **then**
 return $\text{DDmin}(\Delta_j, 2)$;

for $j \in \llbracket 1, g \rrbracket$ **do**

if $\text{test}(\Delta - \Delta_j) = \text{X}$ **then**
 return $\text{DDmin}(\Delta - \Delta_j, g - 1)$;
 $\Delta \leftarrow \Delta - \Delta_j$;
 $g \leftarrow g - 1$;

if $g = \#\Delta$ **then** **return** Δ ;

return $\text{DDmin}(\Delta, \min(2g, \#\Delta))$;



Remarques :

- ◆ $\text{assert}(\text{test}(\{\}) = \checkmark)$
- ◆ $\text{assert}(\text{test}(\Delta) = \text{X})$
- ◆ $\text{split}(\Delta - \Delta_j, g - 1) = \{\Delta_i \mid i \neq j\}$
- ◆ on peut commencer avec une granularité supérieure à 2.
- ◆ on peut ignorer la première boucle.
- ◆ l'implémentation de Zeller casse la récursion.
- ◆ on a besoin d'un cache.

RDDMIN

Recursive DDmin (rDDmin)

Data : Δ : search space

$\Delta_{\text{current}} \leftarrow \Delta$;

$\nabla_{\text{rddmin}} \leftarrow \{\}$;

while $\text{test}(\Delta_{\text{current}}) = \text{X}$ **do**

$\Delta_{\text{ddmin}} \leftarrow \text{DDmin}(\Delta_{\text{current}})$;

$\nabla_{\text{rddmin}} \leftarrow \nabla_{\text{rddmin}} \cup \{\Delta_{\text{ddmin}}\}$;

$\Delta_{\text{current}} \leftarrow \Delta_{\text{current}} - \Delta_{\text{ddmin}}$;

return ∇_{rddmin} ;

Propriétés :

◆ On note $\Delta_{\text{ddcmp}} = \Delta - \bigcup_{\Delta_{\text{ddmin}} \in \nabla_{\text{rddmin}}} \Delta_{\text{ddmin}}$

◆ A priori $\Delta_{\text{ddmax}} \neq \Delta_{\text{ddcmp}}$

◆ A priori $\nabla_{\text{rddmin}} \neq \{\Delta_{1\text{-min}} \subset \Delta \mid \Delta_{1\text{-min}} \text{ étant 1-minimal}\}$

Intérêts :

- ◆ Résultats en ligne
- ◆ Vision globale
- ◆ Peut prendre en compte des heuristiques

Difficultés :

- ◆ Non-parallélisable
- ◆ Résultats dépendants de l'ordre

Impact de l'évaluation stochastique

$test_n$

Data : Δ : search space

Data : n : number of sample

```
for  $j \in \llbracket 1, n \rrbracket$  do
  if  $test\_verrou(\Delta) = \text{X}$  then
    return  $\text{X}$ ;
return ✓ ;
```

Avec n trop faible :

- ◆ Un ensemble dmin de taille 1 est nécessairement 1-min.
- ◆ Un ensemble dmin peut ne pas être assez séparé.
- ◆ L'ensemble dd-cmp peut être trop grand.

Implémentation dans DD_stoch.py

- ◆ Python avec l'utilisation de l'implémentation de Zeller (Il ne reste plus grand chose)
- ◆ Généricité de l'espace de recherche (gestion inclusion/exclusion)
- ◆ Gestion des approches stochastiques (*ie* nombre d'échantillons et graine)
- ◆ Gestion du cache (On a renoncé au cache de Zeller)
- ◆ Ajout algorithmique
- ◆ Parallélisme

Répertoire de cache et de résultats

Nécessité du cache :

- ◆ le delta-debug par Zeller implique plusieurs fois la même configuration.
- ◆ cela stabilise les méthodes stochastiques.

Organisation du répertoire de cache :

```
> ls dd.line
0da42604aec3997e88cc84b7bda5ce4b
1e33d3536f31b7ef73714b25590e8354
3f52aca139a81e9cd961125991afe4a4
503f0ac82ebfcaa5c1f66aa687e9d9b8
566428eb20445a8475a3e48390a17880
66bf7def1ce4225779ec7d6123b353b9
88f45d1e87e27fb4e206afe547705b16
98dfe11b9d4ba7a2bc9f53935eb9b678
cmd.last
d41d8cd98f00b204e9800998ecf8427e
d4e40cc964529b8f13ad8a67cabb16e3
ddmin0 -> 566428eb20445a8475a3e48390a17880
ddmin1 -> 98dfe11b9d4ba7a2bc9f53935eb9b678
fb8d2855419be8c2e6c6349381ceddf9
FullPerturbation -> 88f45d1e87e27fb4e206afe547705b16
NoPerturbation -> d41d8cd98f00b204e9800998ecf8427e
rddmin-cmp -> 66bf7def1ce4225779ec7d6123b353b9
```

ref

Répertoire de cache et de résultats

Nécessité du cache :

- ◆ le delta-debug par Zeller implique plusieurs fois la même configuration.
- ◆ cela stabilise les méthodes stochastiques.

Organisation du répertoire de cache :

```
> ls dd.line
Oda42604aec3997e88cc84b7bda5ce4b
1e33d3536f31b7ef73714b25590e8354
3f52aca139a81e9cd961125991afe4a4
503f0ac82ebfcaa5c1f66aa687e9d9b8
566428eb20445a8475a3e48390a17880
66bf7def1ce4225779ec7d6123b353b9
88f45d1e87e27fb4e206afe547705b16
98dfe11b9d4ba7a2bc9f53935eb9b678
cmd.last
d41d8cd98f00b204e9800998ecf8427e
d4e40cc964529b8f13ad8a67cabb16e3
ddmin0 -> 566428eb20445a8475a3e48390a17880
ddmin1 -> 98dfe11b9d4ba7a2bc9f53935eb9b678
fb8d2855419be8c2e6c6349381ceddf9
FullPerturbation -> 88f45d1e87e27fb4e206afe547705b16
NoPerturbation -> d41d8cd98f00b204e9800998ecf8427e
rddmin-cmp -> 66bf7def1ce4225779ec7d6123b353b9

ref
```

```
> ls Oda42604aec3997e88cc84b7bda5ce4b
dd.line.exclude
dd.line.include
dd.run0
dd.run1
dd.run2
```

Répertoire de cache et de résultats

Nécessité du cache :

- ◆ le delta-debug par Zeller implique plusieurs fois la même configuration.
- ◆ cela stabilise les méthodes stochastiques.

Organisation du répertoire de cache :

```
> ls dd.line
0da42604aec3997e88cc84b7bda5ce4b
1e33d3536f31b7ef73714b25590e8354
3f52aca139a81e9cd961125991afe4a4
503f0ac82ebfcaa5c1f66aa687e9d9b8
566428eb20445a8475a3e48390a17880
66bf7def1ce4225779ec7d6123b353b9
88f45d1e87e27fb4e206afe547705b16
98dfe11b9d4ba7a2bc9f53935eb9b678
cmd.last
d41d8cd98f00b204e9800998ecf8427e
d4e40cc964529b8f13ad8a67cabb16e3
ddmin0 -> 566428eb20445a8475a3e48390a17880
ddmin1 -> 98dfe11b9d4ba7a2bc9f53935eb9b678
fb8d2855419be8c2e6c6349381ceddf9
FullPerturbation -> 88f45d1e87e27fb4e206afe547705b16
NoPerturbation -> d41d8cd98f00b204e9800998ecf8427e
rddmin-cmp -> 66bf7def1ce4225779ec7d6123b353b9
ref
```

```
> ls 0da42604aec3997e88cc84b7bda5ce4b
dd.line.exclude
dd.line.include
dd.run0
dd.run1
dd.run2

> ls dd.run0
dd.compare.err
dd.compare.out
dd.return.value
dd.run.err
dd.run.out
res.dat
```


Options liées au cache

-cache=

- ◆ `continue` : utile pour une reprise avec éventuellement des modifications manuelles (suppression des configurations avec une erreur système ou réseau).
- ◆ `clean` : du passé faisons table rase.
- ◆ `rename` : on conserve l'ancien cache dans un répertoire renommé.
- ◆ `rename_keep_result` : on conserve l'ancien cache dans un répertoire renommé, en supprimant les configurations intermédiaires.
- ◆ `keep_run` : comme `continue` mais on relance la comparaison systématiquement.

Options liées aux heuristiques

- ◆ `-rddmin-heuristics-cache=[none,cache,all_cache]`
- ◆ `-rddmin-heuristics-rep=`
- ◆ `-rddmin-heuristics-file=`
- ◆ `-rddmin-heuristics-line-conv`

Cascade algorithmique

- ① Heuristiques provenant des anciens caches :
 - ▶ code avant modification.
 - ▶ même code mais autres cas tests.
 - ▶ delta-debug non-terminé.
- ② Dichotomy split
- ③ Delta-Debug avant un nombre d'échantillons réduit

Nombre d'échantillons réduit

SrDDMin

Data : Δ : search space

Data : $nTab$: number of samples

$\Delta_{current} \leftarrow \Delta$; $\nabla_{rddmin} \leftarrow \{\}$;

foreach $n \in nTab$ **do**

while $test_n(\Delta_{current}) = \text{X}$ **do**

$\Delta_{ddmin} \leftarrow DDMIN_n(current)$;

foreach $nCheck \in nTab$ **with** $nCheck > n$ **do**

$\Delta_{ddmin} \leftarrow DDMIN_{nCheck}(\Delta_{ddmin})$;

$\nabla_{rddmin} \leftarrow \nabla_{rddmin} \cup \{\Delta_{ddmin}\}$;

$\Delta_{current} \leftarrow \Delta_{current} - \Delta_{ddmin}$;

return ∇_{rddmin} ;

- ◆ On crée un biais sur les ensembles ddmin avec un taux fréquent.
- ◆ On peut être amené à augmenter la granularité dans DDMIN plutôt que de réduire la taille de l'espace.
- ◆ Dans la pratique on ajoute un garde fou sur n en fonction du taux d'erreur de $test(\Delta)$.

Dichotomy Split

Data : Δ : search space

Data : g : granularity (default value 2)

```
if test( $\Delta$ ) = ✓ then return {};  
{ $\Delta_1, \dots, \Delta_g$ }  $\leftarrow$  split( $\Delta, g$ ) ;  
 $\nabla \leftarrow \{ \}$  ;  
subsetFailed  $\leftarrow$  False ;  
for  $j \in \llbracket 1, g \rrbracket$  do  
    if test( $\Delta_j$ ) = ✗ then  
        subsetFailed = True ;  
         $\nabla \leftarrow \nabla \cup \text{DichotomySplit}(\Delta_j, g)$ ;  
if subsetFailed then return  $\nabla$  ;  
else return { $\Delta$ }
```

- ◆ Trouve les ensembles 1-minimal de taille 1.
- ◆ Ne gère que des ensembles contigus (et pas tous).
- ◆ Plus simplement parallélisable que le delta-debug.

DD depuis Dichotomy split

rDDMinFromSplit

Data : Δ : search space

Data : ∇_{split} : split list

Data : n :number of samples

$\Delta_{current} \leftarrow \Delta$; $\nabla_{rddmin} \leftarrow \{\}$;

foreach $\Delta_{split} \in \nabla_{split}$ **with** $\Delta_{split} \subset \Delta_{current}$ **do**

$\nabla_{dd_split} \leftarrow algo_rddmin_n(\Delta_{split})$;
 $\nabla_{rddmin} \leftarrow \nabla_{rddmin} \cup \nabla_{dd_split}$;
 $\Delta_{current} \leftarrow \Delta_{current} - (\cup \nabla_{dd_split})$;

return $\nabla_{rddmin} \cup algo_rddmin_n(\Delta_{current})$;

DD depuis les heuristiques

rDDMinWithHeuristic

Data : Δ : search space

Data : $\nabla_{heuristics}$: list of heuristic

Data : n : number of samples

$\Delta_{current} \leftarrow \Delta$; $\nabla_{rddmin} \leftarrow \{\}$;

foreach $\Delta_{heuristic} \in \nabla_{heuristics}$ *with* $\Delta_{heuristic} \subset \Delta_{current}$ **do**

if $test_n(\Delta_{heuristic}) = \text{X}$ **then**

$is1minimal \leftarrow True$;

foreach $\delta \in \Delta_{heuristic}$ **do**

if $test_n(\Delta_{heuristic} - \delta) = \text{X}$ **then**

$is1minimal \leftarrow False$; **break** ;

if $is1minimal$ **then**

$\nabla_{rddmin} \leftarrow \nabla_{rddmin} \cup \{\Delta_{heuristic}\}$;

$\Delta_{current} \leftarrow \Delta_{current} - \Delta_{heuristic}$;

else

$\nabla_{heuristic} \leftarrow algo_rddmin_n(\Delta_{heuristic})$;

$\nabla_{rddmin} \leftarrow \nabla_{rddmin} \cup \nabla_{heuristic}$;

$\Delta_{current} \leftarrow \Delta_{current} - (\cup \nabla_{heuristic})$;

return $\nabla_{rddmin} \cup algo_rddmin_n(\Delta_{current})$;

Delta-Debug

Parallélisme actuel

Implémentation

- ◆ Python: `concurrent.futures.ThreadPoolExecutor`
- ◆ Une distribution peut être implémentée dans le script `run`.
- ◆ Un calcul commencé est fini (pour garder un cache cohérent).

Niveaux de parallélisme

- ① Au niveau de la fonction test
 - ▶ Facile
 - ▶ Pas très efficace quand la fréquence des erreurs est élevée.
- ② Au niveau du split
 - ▶ “semble marcher” : algo au prochain slide
 - ▶ On peut jouer avec la granularité
- ③ Au niveau du delta-debug.
 - ▶ Ne marche pas : algo non intégré

Parallélisme split

```
 $\nabla_{current} \leftarrow \{\Delta\}; \nabla_{res} \leftarrow \{\};$   
while  $\#\nabla_{current} > 0$  do  
   $blocSize = \min(\lceil nbProc/granularity \rceil, \#\nabla_{current})$  ;  
   $\nabla_{bloc} \leftarrow \nabla_{current}[1 : blocSize]$  ;  
  foreach  $i \in \llbracket 1, \#\nabla_{bloc} \rrbracket$  do  
     $cijTab[i] \leftarrow split(\nabla_{bloc}[i], \min(granularity, \#\Delta_i))$  ;  
     $testijTab \leftarrow testTab_{nbRun}(cijTab)$  ;  
     $\nabla_{current}^{bloc} = \{\}$  ;  
    foreach  $i \in \llbracket 1, \#\nabla_{bloc} \rrbracket$  do  
       $subsetFailed \leftarrow False$ ;  
      foreach  $j \in \llbracket 1, \min(granularity, \#\nabla_{bloc}[i]) \rrbracket$  do  
         $\Delta_{ij} \leftarrow cijTab[i][j]$ ;  
        if  $testijTab[i][j] = \text{X}$  then  
           $subsetFailed \leftarrow True$ ;  
          if  $\#conf = 1$  then  $\nabla_{res} \leftarrow \nabla_{res} \cup \{\Delta_{ij}\}$ ;  
          else  $\nabla_{current}^{bloc} \leftarrow \nabla_{current}^{bloc} \cup \{\Delta_{ij}\}$ ;  
        if not subsetFailed then  $\nabla_{res} \leftarrow \nabla_{res} \cup \{\nabla_{bloc}[i]\}$  ;  
   $\nabla_{current} \leftarrow \nabla_{current}^{bloc} \cup \nabla_{current}[blocSize + 1 : *]$ ;  
return  $\nabla_{res}$  ;
```

Delta-Debug

Et les runtime dans tous ça!

L'espoir

- ◆ Réduire les synchronisations.
- ◆ Meilleure adaptabilité aux données.
- ◆ Outils de profiling intégré.
- ◆ Meilleure expressivité pour simplifier le développement.
- ◆ Gestion des priorités moins implicite.
- ◆ Gestion dynamique des ressources.
- ◆ Enchaînement entre types de delta-debug.

Les difficultés

- ◆ Conserver un mode dégradé sans nouvelle dépendance.
- ◆ Quand et comment placer les synchronisations?
- ◆ Quelle signification pour rddmin?
- ◆ Définition d'une tâche/tâche hiérarchique et quelle interaction avec le cache?
- ◆ Grande variété de comportements.

Les questions ouvertes

- ◆ Faut-il attaquer le problème maintenant ou clarifier l'algorithmique avant?
 - ▶ Exécution spéculative ou granularité adaptative?
 - ▶ Adaptation de dmin pour le contexte rddmin.
 - ▶ Adaptation dynamique du nombre d'échantillons (granularité, sous-espaces / sous-espaces complémentaires).
 - ▶ Génération de données supplémentaires pour des heuristiques (exemple nombre d'opérations par ligne)
- ◆ Faut-il regarder des algorithmes utilisant la distance à la référence plutôt qu'un seuil?